Geometric Brownian motion and it's applications in stock prices

Introduction to Brownian Motion

Brownian motion, denoted as $W(t)$, is a fundamental stochastic process defined by the following properties:

1. $W(0) = 0$
2. $W(t)$ has independent increments: $W(t) - W(s)$ is independent of $W(u)$ for u < s
3. $W(t) - W(s) \sim N(0, t-s)$ for $0 \leq s < t$, meaning it is normally distributed with mean 0 and variance t − s
4. W(t) has continuous paths almost surely

These properties make Brownian motion a central tool in modeling random phenomena, particularly in finance.

Definition of Geometric Brownian Motion

Geometric Brownian Motion (GBM) is used to model the evolution of stock prices. Formally, a stock price S(t) follows a GBM if it satisfies the stochastic differential equation (SDE):

$$dS(t) = \mu S(t)dt + \sigma S(t)dW(t)$$

where:

- μ is the drift coefficient (mean rate of return)
- σ is the volatility coefficient (standard deviation of the return)
- W(t) is a standard Brownian motion

Derivation of the GBM SDE

To derive the SDE for GBM, consider the proportional change in stock price over time. The assumption is that the proportional change in S(t) is normally distributed. Thus, we write:

$$\frac{dS(t)}{S(t)} = \mu\, dt + \sigma\, dW(t)$$

Rearranging, we get:

$$dS(t) = \mu S(t)dt + \sigma S(t)dW(t)$$

This equation expresses that the change in stock price $dS(t)$ consists of a deterministic part $\mu S(t)$ and a stochastic part $\sigma S(t)dW(t)$.

Solving the SDE using Itô's Lemma

Itô's Lemma is a fundamental result in stochastic calculus used to find the differential of a function of a stochastic process. For a function $f(S(t), t)$,Itô's Lemma states:

$$df = \left(\frac{\partial f}{\partial t} + \mu S(t)\frac{\partial f}{\partial S} + \frac{1}{2}\sigma^2 S(t)^2 \frac{\partial^2 f}{\partial S^2}\right) dt + \sigma S(t)\frac{\partial f}{\partial S} dW(t)$$

Consider $f(S(t), t) = \ln S(t)$ Then:

- $\frac{\partial f}{\partial t} = 0$
- $\frac{\partial f}{\partial S} = \frac{1}{S(t)}$
- $\frac{\partial^2 f}{\partial S^2} = -\frac{1}{S(t)^2}$

Applying Itô's Lemma to $f\big(S(t)\big) = \ln S(t)$:

$$d(\ln S(t)) = \frac{1}{S(t)} dS(t) - \frac{1}{2}\frac{1}{S(t)^2}(dS(t))^2$$

Substitute $dS(t) = \mu S(t)dt + \sigma S(t)dW(t)$:

$$d(\ln S(t) = \frac{1}{S(t)}\ (\mu S(t)dt + \sigma S(t)dW(t) - \frac{1}{2}\frac{1}{S(t)^2}\big(\sigma S(t)\big)^2 dt$$

Simplifying, we get:

$$d(\ln S(t)) = \ \mu dt + \sigma\ dW(t) - \frac{1}{2}\sigma^2 dt$$

Thus:

$$d(\ln S(t)) = (\mu - \frac{\sigma^2}{2})dt + \sigma\ dW(t)$$

Integrate both sides from 0 to t:

$$\ln S(t) = \ln S(0) + \left(\mu - \frac{\sigma^2}{2}\right)t + \sigma\ W(t)$$

Exponentiating both sides gives the solution to the SDE:

$$S(t) = S(0)e^{\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma\ W(t)}$$

Comparison with Standard Brownian Motion

Standard Brownian motion W(t) and Geometric Brownian Motion S(t) have distinct characteristics:

1. **Nature of Change**:
   - **Brownian Motion**: Represents additive changes.
   - **Geometric Brownian Motion**: Represents multiplicative changes due to exponentiation.
2. **Distribution**:
   - **Brownian Motion**: Values of W(t) are normally distributed with mean 0 and variance t.
   - **Geometric Brownian Motion**: S(t) is log-normally distributed, ensuring positive values.
3. **Path Properties**:
   - **Brownian Motion**: Paths can take any real value, including negative values.
   - **Geometric Brownian Motion**: Paths remain strictly positive, reflecting realistic stock price behavior.
4. **Financial Relevance**:
   - **Brownian Motion**: Suitable for modeling raw stochastic processes.
   - **Geometric Brownian Motion**: Used specifically to model asset prices in financial markets.

Application of Geometric Brownian Motion in Simulating Stock Prices

Geometric Brownian Motion (GBM) is extensively used in finance to simulate the future paths of stock prices. This simulation helps investors and analysts understand the range of potential outcomes and the probability distribution of future prices, aiding in risk assessment and investment decision-making.

Code for Simulating Stock Prices Using GBM

Below is the Python code to simulate stock prices using Geometric Brownian Motion, along with an explanation of each part:

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
S0 = 150  # Initial stock price
mu = 0.1  # Drift coefficient (expected return)
sigma = 0.2  # Volatility (standard deviation of returns)
T = 1.0  # Time horizon (in years)
N = 300  # Number of time steps (e.g., trading days in a year)
dt = T / N  # Time step
num_simulations = 1 2 # Number of simulated paths

# Generate paths
np.random.seed(42)  # For reproducibility
t = np.linspace(0, T, N)
S = np.zeros((N, num_simulations))
S[0] = S0

for i in range(1, N):
    Z = np.random.standard_normal(num_simulations)
    S[i] = S[i-1] * np.exp((mu - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z)

# Plot results
```

```
plt.figure(figsize=(10, 6))
for j in range(num_simulations):
    plt.plot(t, S[:, j], lw=1.5, alpha=0.6)
plt.title('Simulated Stock Price Paths using Geometric Brownian Motion')
plt.xlabel('Time (years)')
plt.ylabel('Stock Price')
plt.grid(True)
plt.show()
```

Explanation of the code :

- **Parameters**

```
S0 = 150  # Initial stock price
mu = 0.1  # Drift coefficient (expected return)
sigma = 0.2  # Volatility (standard deviation of returns)
T = 1.0  # Time horizon (in years)
N = 300  # Number of time steps (e.g., trading days in a year)
dt = T / N  # Time step
num_simulations = 12  # Number of simulated paths
```

  - `S0`: Initial stock price.
  - `mu`: Drift coefficient representing the expected return.
  - `sigma`: Volatility representing the standard deviation of returns.
  - `T`: Total time horizon for the simulation.
  - `N`: Number of time steps (e.g., trading days in a year).
  - `dt`: Time step size (`T/N`).
  - `num_simulations`: Number of simulated paths.

- **Initialize variables**

```
np.random.seed(42)  # For reproducibility
t = np.linspace(0, T, N)
S = np.zeros((N, num_simulations))
S[0] = S0
```

  - `np.random.seed(42)`: Sets the random seed for reproducibility of results.
  - `t`: Array of time points from 0 to `T`.
  - `S`: Array to store the simulated stock prices. Initialized to zero with the first row set to the initial stock price `S0`.

- **Simulate stock prices**

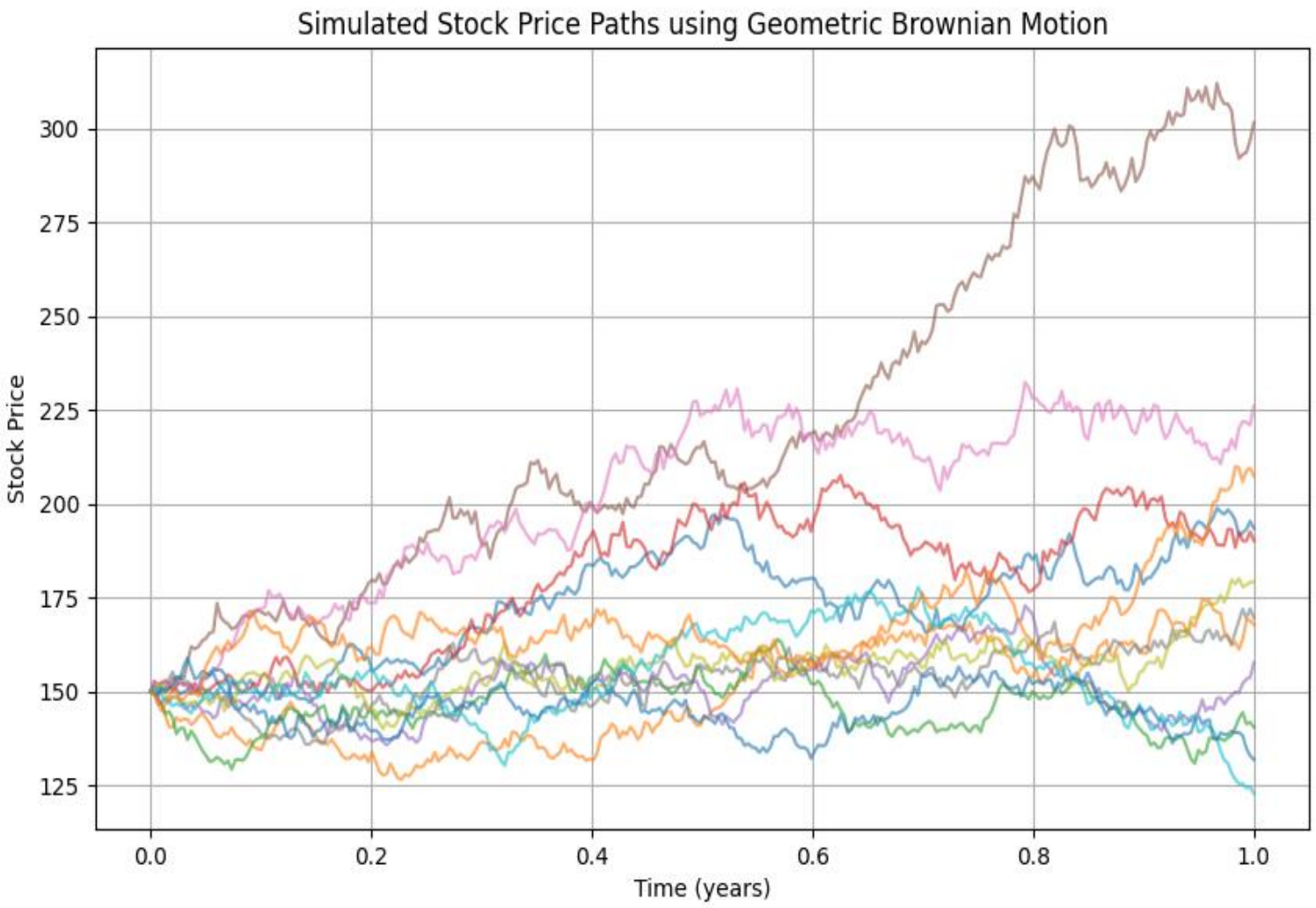
```
for i in range(1, N):
    Z = np.random.standard_normal(num_simulations)
    S[i] = S[i-1] * np.exp((mu - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z)
```

  - Loop through each time step from 1 to N.
  - Z: Array of random standard normal variables.
  - Update the stock price at each time step using the GBM formula:

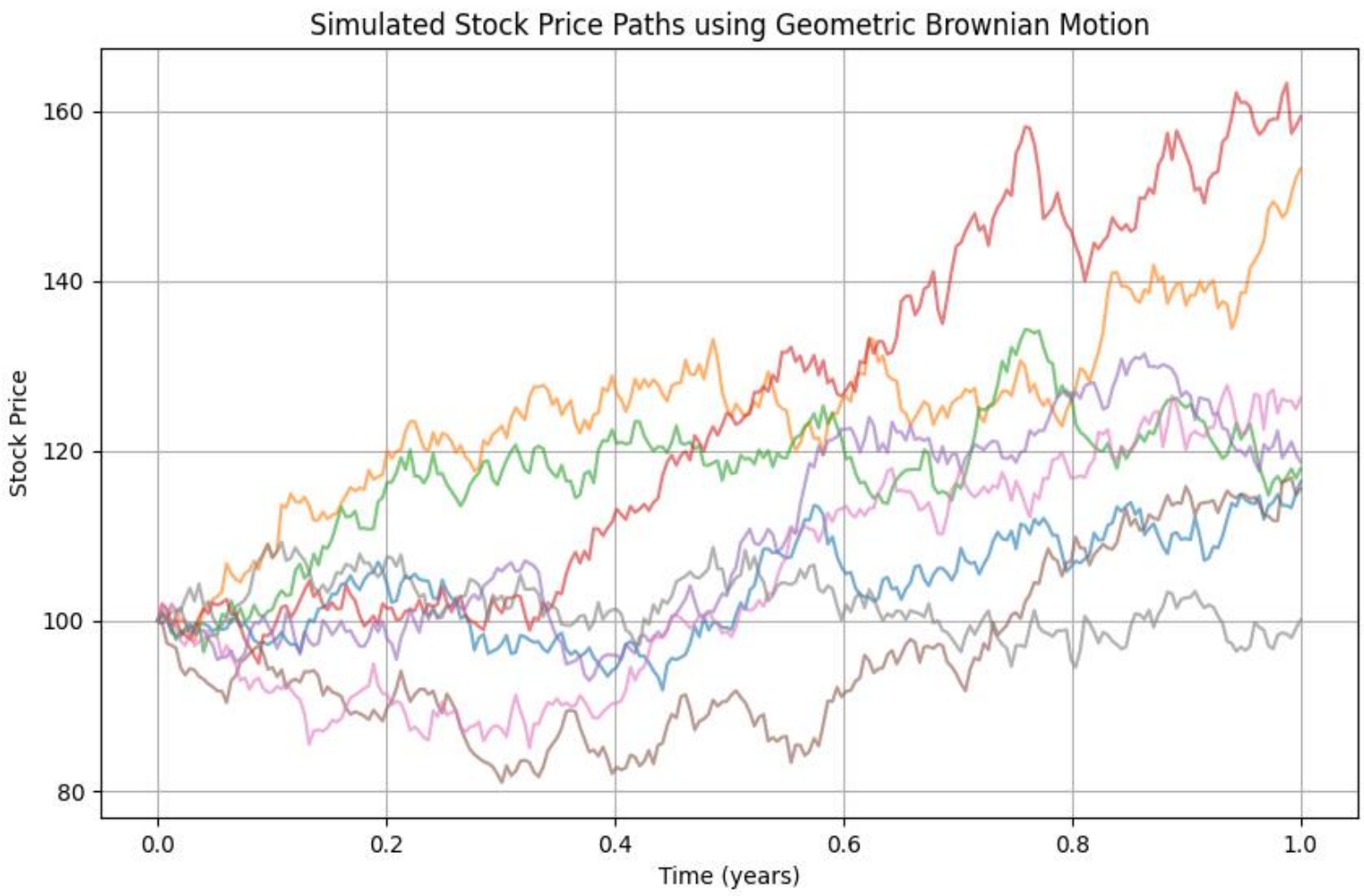
$$S(t + \Delta T) = S(t)\, \mathrm{e}^{\left(\mu - \frac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}Z}$$

This formula ensures that the stock price follows a log-normal distribution, capturing both the drift and volatility of the price changes.

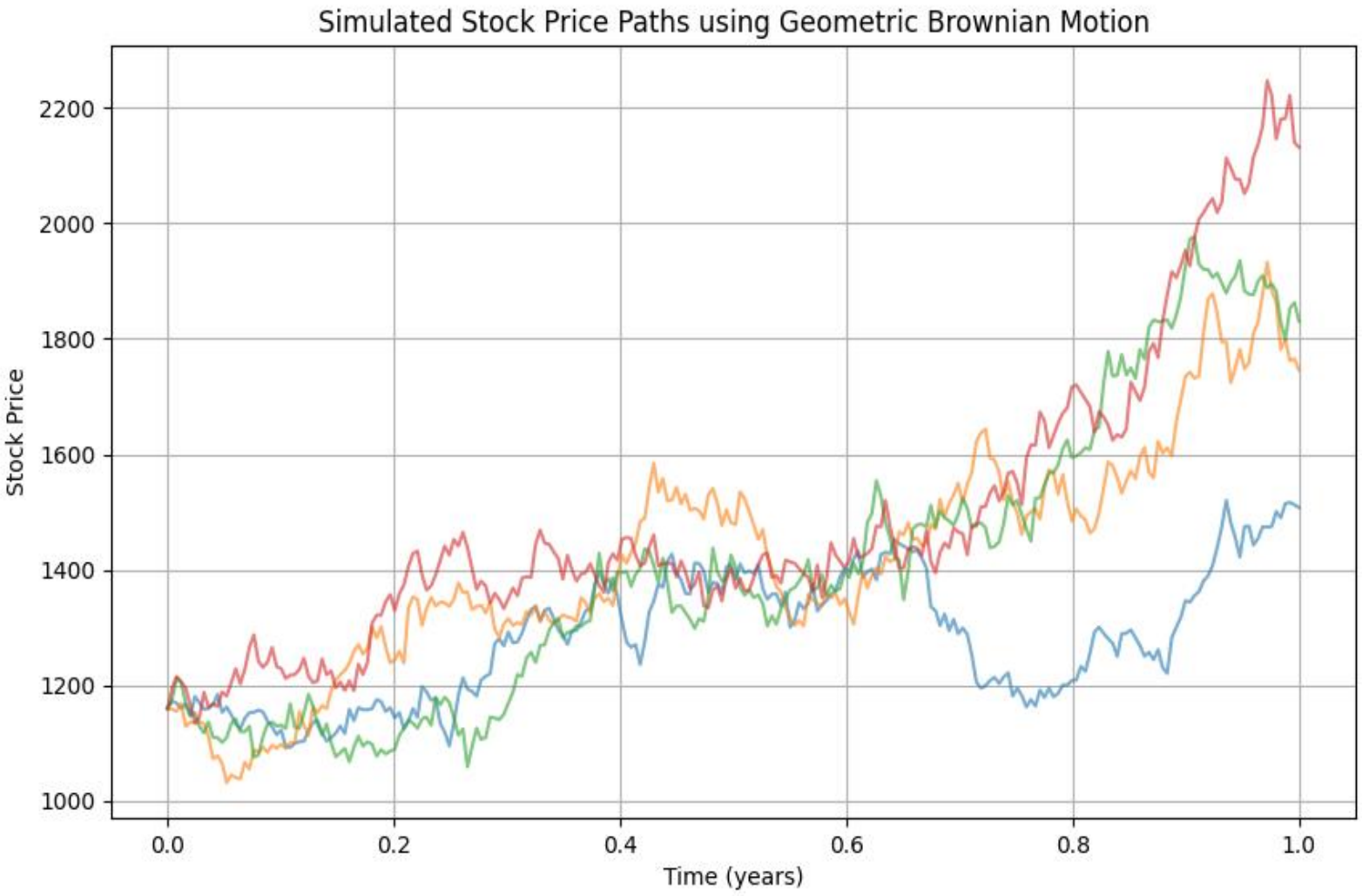
- **Plotting price paths for different initial values(parameters)**




```
      # Parameters
S0 = 150  # Initial stock price
mu = 0.1  # Drift coefficient (expected return)
sigma = 0.2  # Volatility (standard deviation of returns)
T = 1.0  # Time horizon (in years)
N = 300  # Number of time steps (e.g., trading days in a year)
dt = T / N  # Time step
num_simulations = 12  # Number of simulated paths
```




```
# Parameters
S0 = 100  # Initial stock price
mu = 0.1  # Drift coefficient (expected return)
sigma = 0.2  # Volatility (standard deviation of returns)
T = 1.0  # Time horizon (in years)
N = 250  # Number of time steps (e.g., trading days in a year)
dt = T / N  # Time step
num_simulations = 8  # Number of simulated paths
```




```
# Parameters
S0 = 1160  # Initial stock price
mu = 0.4  # Drift coefficient (expected return)
sigma = 0.3  # Volatility (standard deviation of returns)
T = 1.0  # Time horizon (in years)
N = 250  # Number of time steps (e.g., trading days in a year)
dt = T / N  # Time step
num_simulations = 4  # Number of simulated paths
```